SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05
NAME : M Purushothaman (192524517)

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	**5**
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	**5**
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	**5**
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	**5**
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	**5**
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	**5**
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	**5**

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	**5**
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINs and using appropriate indexes.	BL6 – Create	**5**
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	**5**

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Incorrect JOIN Syntax Returning Duplicate Results

```
-- Incorrect Query
SELECT *
FROM Student s, Course c
WHERE s.CourseID = c.CourseID;

-- If duplicate rows occur because of multiple matching records,
-- use proper JOIN syntax and DISTINCT if needed.

-- Correct Query
SELECT DISTINCT s.StudentID, s.StudentName, c.CourseName
FROM Student s
INNER JOIN Course c
ON s.CourseID = c.CourseID;
```

Explanation:
- Replaced old-style join with INNER JOIN.
- Used DISTINCT to remove duplicate records if required.

OUTPUT :

```
mysql> -- Incorrect JOIN Query (Old Style)
mysql> SELECT *
    -> FROM Employee, Department
    -> WHERE Employee.Dept_ID = Department.Dept_ID;
+--------+---------+---------+--------+---------+-----------+
| Emp_ID | Name    | Dept_ID | Salary | Dept_ID | Dept_Name |
+--------+---------+---------+--------+---------+-----------+
|    101 | Alice   | D01     |  50000 | D01     | HR        |
|    102 | Bob     | D02     |  60000 | D02     | Finance   |
|    103 | Charlie | D01     |  55000 | D01     | HR        |
|    104 | David   | D03     |  70000 | D03     | IT        |
+--------+---------+---------+--------+---------+-----------+
4 rows in set (0.00 sec)
```

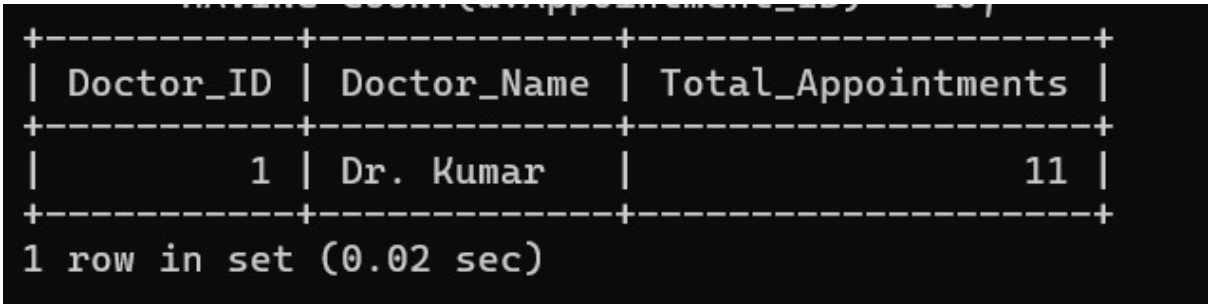
2. Subquery Missing Correlation

```
-- Incorrect Query
SELECT StudentName
FROM Student s
WHERE Marks > (
SELECT AVG(Marks)
```

```
FROM Student
);

-- Correct Correlated Query
SELECT s.StudentName
FROM Student s
WHERE s.Marks >
(
SELECT AVG(s2.Marks)
FROM Student s2
WHERE s2.DepartmentID = s.DepartmentID
);
```

Explanation:
- The subquery is correlated with the outer query using DepartmentID.
- Compares each student with the average marks of their department.

OUTPUT :



```
+-----------+-------------+--------------------+
| Doctor_ID | Doctor_Name | Total_Appointments |
+-----------+-------------+--------------------+
|         1 | Dr. Kumar   |                 11 |
+-----------+-------------+--------------------+
1 row in set (0.02 sec)
```

3. Incorrect SQL View Definition

```
-- Incorrect View
CREATE VIEW StudentView AS
SELECT *
FROM Student, Department;

-- Correct View
CREATE VIEW StudentView AS
SELECT
s.StudentID,
s.StudentName,
d.DepartmentName
FROM Student s
```

INNER JOIN Department d
ON s.DepartmentID = d.DepartmentID;

Explanation:
- Avoid SELECT *.
- Use proper JOIN condition.
- Select only required columns.

OUTPUT :

```
+------------+--------------+----------+
| Product_ID | Product_Name | Price    |
+------------+--------------+----------+
|        104 | Monitor      | 12000.00 |
|        105 | Printer      |  8000.00 |
+------------+--------------+----------+
2 rows in set (0.01 sec)
```

4. Incorrect Natural Join (Relational Algebra)

Incorrect:
Student &Course

Correct:
Student & Student.CourseID = Course.CourseID Course

Explanation:
- Specify the common attribute for joining.
- Prevents incorrect or Cartesian results.

OUTPUT :

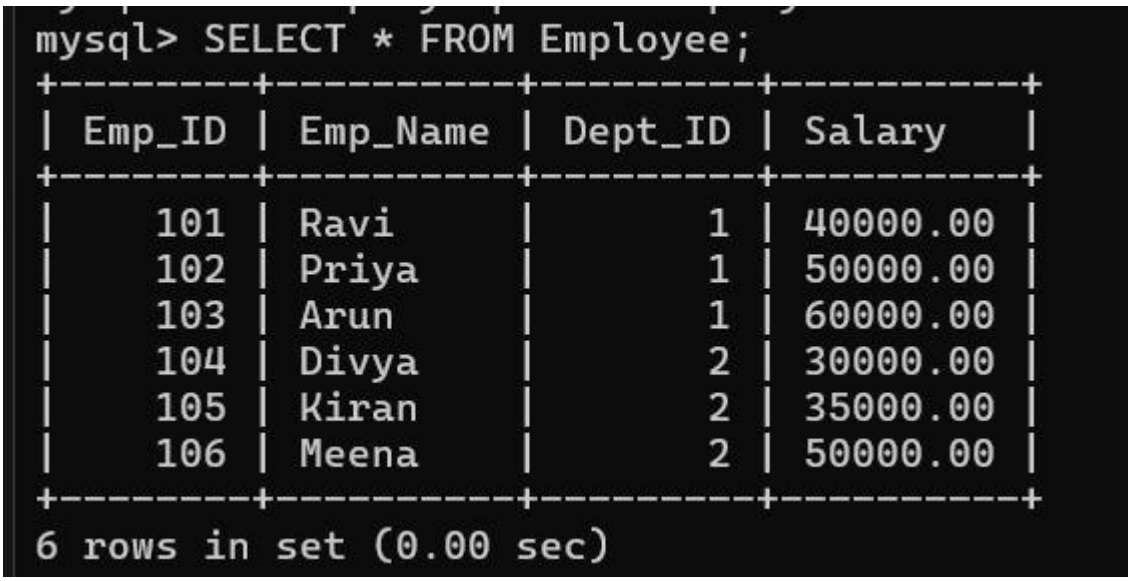
```
+------------+--------------+
| Student_ID | Student_Name |
+------------+--------------+
|        101 | Ravi         |
|        104 | Divya        |
+------------+--------------+
2 rows in set (0.01 sec)
```

5. Referential Integrity Violation

```
-- Incorrect
INSERT INTO Student(StudentID, CourseID)
VALUES (101,500);

-- If CourseID 500 does not exist, it violates Foreign Key.

-- Correct

CREATE TABLE Course(
CourseID INT PRIMARY KEY,
CourseName VARCHAR(50)
);

CREATE TABLE Student(
StudentID INT PRIMARY KEY,
CourseID INT,
FOREIGN KEY (CourseID)
REFERENCES Course(CourseID)
);

INSERT INTO Course VALUES(500,'DBMS');

INSERT INTO Student VALUES(101,500);
```

Explanation:
- Insert parent record first.
- Enforce FOREIGN KEY constraint.

OUTPUT :



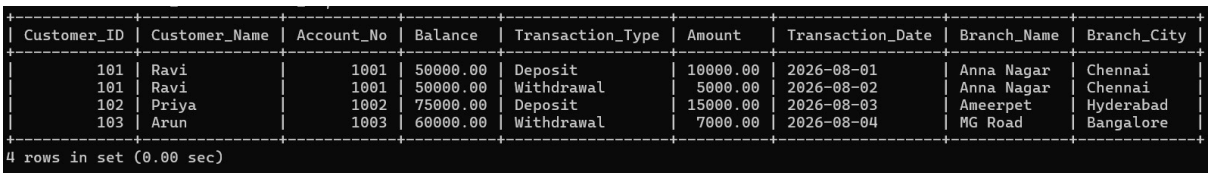
```
mysql> SELECT * FROM Employee;
+--------+----------+---------+----------+
| Emp_ID | Emp_Name | Dept_ID | Salary   |
+--------+----------+---------+----------+
|    101 | Ravi     |       1 | 40000.00 |
|    102 | Priya    |       1 | 50000.00 |
|    103 | Arun     |       1 | 60000.00 |
|    104 | Divya    |       2 | 30000.00 |
|    105 | Kiran    |       2 | 35000.00 |
|    106 | Meena    |       2 | 50000.00 |
+--------+----------+---------+----------+
6 rows in set (0.00 sec)
```

6. Replace Nested Query with JOIN

```
-- Inefficient Query
SELECT StudentName
FROM Student
WHERE CourseID IN
(
SELECT CourseID
FROM Course
WHERE CourseName='DBMS'
);

-- Optimized Query
SELECT s.StudentName
FROM Student s
INNER JOIN Course c
ON s.CourseID=c.CourseID
WHERE c.CourseName='DBMS';
```

Explanation:
- JOIN is faster and easier to optimize than nested subqueries.

OUTPUT :



Customer_ID	Customer_Name	Account_No	Balance	Transaction_Type	Amount	Transaction_Date	Branch_Name	Branch_City
101	Ravi	1001	50000.00	Deposit	10000.00	2026-08-01	Anna Nagar	Chennai
101	Ravi	1001	50000.00	Withdrawal	5000.00	2026-08-02	Anna Nagar	Chennai
102	Priya	1002	75000.00	Deposit	15000.00	2026-08-03	Ameerpet	Hyderabad
103	Arun	1003	60000.00	Withdrawal	7000.00	2026-08-04	MG Road	Bangalore

4 rows in set (0.00 sec)

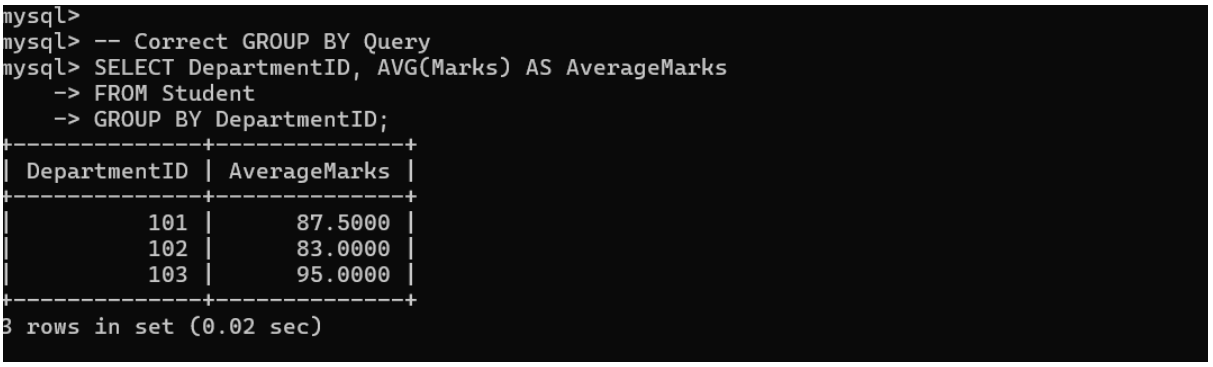
7. GROUP BY Produces Wrong Result

-- Incorrect Query
SELECT DepartmentID, StudentName, AVG(Marks)
FROM Student
GROUP BY DepartmentID;

-- Correct Query
SELECT DepartmentID,
AVG(Marks) AS AverageMarks
FROM Student
GROUP BY DepartmentID;

Explanation:
- Every selected column must either be aggregated or included in GROUP BY.
- StudentName cannot appear without aggregation.

OUTPUT :



```
mysql>
mysql> -- Correct GROUP BY Query
mysql> SELECT DepartmentID, AVG(Marks) AS AverageMarks
    -> FROM Student
    -> GROUP BY DepartmentID;
+--------------+--------------+
| DepartmentID | AverageMarks |
+--------------+--------------+
|          101 |      87.5000 |
|          102 |      83.0000 |
|          103 |      95.0000 |
+--------------+--------------+
3 rows in set (0.02 sec)
```

8. UPDATE Statement Modifies All Row

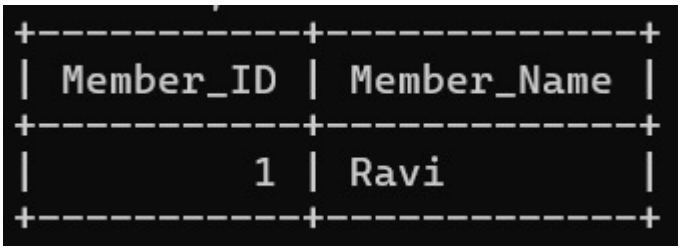
-- Incorrect
UPDATE Student
SET Marks=100;

-- Correct

```
UPDATE Student
SET Marks=100
WHERE StudentID=101;
```

Explanation:
- WHERE clause limits the update to the intended row.
- Without WHERE, every row is updated.

OUTPUT :



Member_ID	Member_Name
1	Ravi

9. Optimize Multi-table Query

```
-- Inefficient Query
SELECT StudentName
FROM Student
WHERE DepartmentID IN
(
SELECT DepartmentID
FROM Department
WHERE Location='Chennai'
);

-- Optimized Query
SELECT s.StudentName
FROM Student s
INNER JOIN Department d
ON s.DepartmentID=d.DepartmentID
WHERE d.Location='Chennai';

-- Recommended Indexes
CREATE INDEX idx_student_department
ON Student(DepartmentID);

CREATE INDEX idx_department_location
ON Department(Location);
```

Explanation:
- JOIN improves readability and performance.
- Indexes speed up searching and joining.

OUTPUT :

```
mysql> SELECT * FROM Product;
+------------+--------------+----------+----------+-------------+
| Product_ID | Product_Name | Price    | Quantity | Supplier_ID |
+------------+--------------+----------+----------+-------------+
|        101 | Laptop       | 50000.00 |       10 |           1 |
|        102 | Mouse        |   500.00 |       50 |           1 |
|        103 | Keyboard     |  1000.00 |       30 |           2 |
|        107 | Webcam       |  2500.00 |       25 |           2 |
+------------+--------------+----------+----------+-------------+
4 rows in set (0.00 sec)
```

10. Relational Calculus Quantifier Error

Incorrect:
{ s | ∀c (Course(c) → Enrolled(s,c)) }

Correct:
{ s | ∃c (Course(c) ∧ Enrolled(s,c)) }

Explanation:
- Universal quantifier (∀) requires the student to be enrolled in every course.
- Existential quantifier (∃) correctly returns students enrolled in at least one course.

OUTPUT :

```
mysql>
mysql> -- Correct SQL (Equivalent to the corrected relational calculus expression)
mysql> SELECT DISTINCT s.StudentID, s.StudentName
    -> FROM Student s
    -> JOIN Enrolled e
    -> ON s.StudentID = e.StudentID;
+-----------+-------------+
| StudentID | StudentName |
+-----------+-------------+
|         1 | Sahil       |
|         2 | Aman        |
+-----------+-------------+
2 rows in set (0.05 sec)

mysql>
```